Python Engineering - Class 10

Advanced Functions | EX18 - Flexible Function Calls

Class objective: Understand different ways to pass arguments, unpack structures and treat functions as objects.

This document is the base exercise guide. It is not a full solution to copy: it organizes the problem, implementation steps, reference values and required validation.

Path

```
01_python_bootcamp/
`-- week_03_modules_and_coordinates/class_10_advanced functions/
    |-- notes/
    `-- exercices/
```

Required workflow

```
PROBLEM
INPUTS
OUTPUTS
EQUATIONS
SPECIAL CASES
DO I NEED ITERATION?
ALGORITHM
VALIDATION
```

Before coding, write these eight sections. The goal is to separate problem reasoning from syntax.

EX18 - Advanced Arguments and Functions as Objects

Problem: Practice positional, keyword, default, *args, **kwargs, unpacking and functions passed as arguments.

Reference inputs: config dictionaries, measurement sequences, add/multiply functions

Equations / relationship: specific energy g*h + 0.5*v^2; averages; simple operations

Expected check: *args is a tuple; **kwargs is a dict; **config keys must match function parameter names; add vs add() distinction.

Implementation guide

- 1. Call with positional/keyword arguments.
- 2. Use default g.
- 3. Create *args.
- 4. Create **kwargs.
- 5. Unpack list/dict.
- 6. Pass add or multiply to apply_operation.

Before running

Predict sign, order of magnitude, data type and any relevant shape. Then compare the prediction with the actual output.

Final validation

Answer these questions as comments at the end of the exercise. The goal is not memorized wording but demonstrating that you can explain the code flow.

- Difference between positional and keyword arguments.
- What happens if a default argument is omitted?
- What type is *args inside a function?
- What type is **kwargs?
- What does f(*values) mean?
- Why must config keys match parameters in f(**config)?
- Difference between add and add().

Completion criterion

The class is complete when the code runs, the reference checks make sense, you can explain every block without reading a solution, and you can modify at least one input while predicting the expected effect.